SiteProof: Database and ETL

Software Engineering in Construction Information Systems (CT5805701), NTUST

A. Samuel Arzamendia M11405807 - Po-Hao Cheng (鄭博浩) M11505503

November 11, 2026

1. Introduction

In this assignment we created the database of SiteProof in MySQL and loaded the data given in class (companies.tsv, materials.tsv and transactions.tsv). We used MySQL 8.4 in Docker, like in the class example. All the work was done with SQL: the files were loaded with LOAD DATA INFILE and we did not change them in Excel.

We wrote five SQL scripts and ran them in order:

- **01_schema.sql**: create the database and the tables of our MVP data model.
- **02_extract.sql**: load the three files into "raw" tables, without changes.
- **03_profile.sql**: find the errors in the data.
- **04_transform_load.sql**: clean the data and load it into the final tables.
- **05_verify.sql**: check the result.

The results in section 4 are copied from the output of MySQL (we ran the scripts with the mysql client in verbose mode).

2. Errors in the data

Error	Found (step)	Fixed (step)	What we did
Transaction with a company that does not exist	3.1	4.1	Deleted (2,615 rows)
Transaction with a material that does not exist	3.2	4.2	Deleted (2,497 rows)
Impossible date (1950-1980)	3.3	4.3	Deleted (4,705 rows)
Total price is a copy of the unit price	3.4, 3.5	1.14, 4.10	Total not stored; the view purchase_total calculates it (2,535 rows)
Same material twice with two ids	3.6	4.4, 4.5	Transactions moved to the id we keep, duplicate deleted
Unit written in two ways (piece/Piece, one test/One test)	3.7	4.6	Same spelling for all (12 materials)

Some rows had more than one error, so the number found in step 3 is sometimes higher than the number deleted in step 4 (for example, 2,643 rows have an unknown material, but 146 of them were already deleted in 4.1).

Result: 51,947 - 2,615 - 2,497 - 4,705 = 42,130 purchases. We also loaded 500 companies and 457 materials (458 minus the duplicated one).

3. Decisions

- We only load the data that our platform needs. Company description and comments, the price statistics of the materials and the transaction notes are not used.
- We do not store the total price, because it can be calculated from quantity, unit price and discount (3rd normal form). This also fixes the wrong totals.
- Categories: items that start with "Product" are Material; items paid by hour, day or trip are Equipment if the name mentions a machine, otherwise Labor; the rest is Other.
- For the duplicated material we keep id 51019678 because it has more price samples.
- The ids are stored as text (CHAR(8)) because they have leading zeros, for example 03028108.
- Steps 5.7 and 5.8 are supposed to fail: they show that the constraints do not accept wrong data.

4. SQL statements and results

Script 1: Create the database and tables

1.1 Delete the database if it already exists, so the whole process can be run again from zero.

```
DROP DATABASE IF EXISTS siteproof;
```

Result:

```
Query OK, 14 rows affected
```

1.2 Create the database (utf8mb4 so symbols like m² are stored correctly).

```
CREATE DATABASE siteproof CHARACTER SET utf8mb4;
```

Result:

```
Query OK, 1 row affected
```

1.3 Use the new database.

```
USE siteproof;
```

1.4 Companies (contractors and suppliers). The id is the 8-digit company id; it is text because it has leading zeros.

```
CREATE TABLE organization (
    org_id   CHAR(8)      PRIMARY KEY,
    org_name VARCHAR(100) NOT NULL,
    org_type VARCHAR(20)  NOT NULL CHECK (org_type IN ('CONTRACTOR', 'SUPPLIER')),
    address  VARCHAR(200),
    phone    VARCHAR(30),
    email    VARCHAR(100)
);
```

Result:

```
Query OK, 0 rows affected
```

1.5 Users of the platform. Each user belongs to one company and has one role.

```
CREATE TABLE user_account (
    user_id   INT AUTO_INCREMENT PRIMARY KEY,
    org_id    CHAR(8)      NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    email     VARCHAR(100) NOT NULL UNIQUE,
    password  VARCHAR(255) NOT NULL,
    role      VARCHAR(20)  NOT NULL CHECK (role IN ('CONTRACTOR_PM', 'SUPPLIER', 'INSPECTOR')),
    FOREIGN KEY (org_id) REFERENCES organization (org_id)
);
```

Result:

```
Query OK, 0 rows affected
```

1.6 Projects of a contractor, with the retention rate of the contract (0.05 = 5%).

```
CREATE TABLE project (
    project_id     INT AUTO_INCREMENT PRIMARY KEY,
    org_id         CHAR(8)      NOT NULL,
    project_name   VARCHAR(100) NOT NULL,
    retention_rate DECIMAL(4,3) NOT NULL CHECK (retention_rate BETWEEN 0 AND 1),
    FOREIGN KEY (org_id) REFERENCES organization (org_id)
);
```

Result:

```
Query OK, 0 rows affected
```

1.7 Categories to search the catalog: labor (man), equipment (machine), material and other.

```
CREATE TABLE material_category (
    category_id   INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(50) NOT NULL UNIQUE
);
```

Result:

```
Query OK, 0 rows affected
```

1.8 Catalog of materials, labor and equipment. Each item belongs to one category.

```
CREATE TABLE material (
    material_id   CHAR(8)      PRIMARY KEY,
    material_name VARCHAR(255) NOT NULL,
    unit          VARCHAR(20)  NOT NULL,
    category_id   INT          NOT NULL,
    FOREIGN KEY (category_id) REFERENCES material_category (category_id)
);
```

Result:

```
Query OK, 0 rows affected
```

1.9 Purchases from the given data. The total price is not stored: it is calculated (step 1.14), so it can never be wrong.

```
CREATE TABLE purchase (
    purchase_id   CHAR(8)       PRIMARY KEY,
    org_id        CHAR(8)       NOT NULL,
    material_id   CHAR(8)       NOT NULL,
    quantity      INT           NOT NULL CHECK (quantity > 0),
    unit_price    DECIMAL(10,2) NOT NULL CHECK (unit_price >= 0),
    discount_rate INT           NOT NULL CHECK (discount_rate BETWEEN 0 AND 100),
    purchase_date DATE          NOT NULL CHECK (purchase_date >= '2000-01-01'),
    FOREIGN KEY (org_id)      REFERENCES organization (org_id),
    FOREIGN KEY (material_id) REFERENCES material (material_id)
);
```

Result:

```
Query OK, 0 rows affected
```

1.10 Delivery record (MVP core): project, supplier, material, quantity, value, time slot and status.

```
CREATE TABLE delivery (
    delivery_id      INT AUTO_INCREMENT PRIMARY KEY,
    project_id       INT           NOT NULL,
    org_id           CHAR(8)       NOT NULL,
    material_id      CHAR(8)       NOT NULL,
    created_by       INT           NOT NULL,
    ordered_quantity DECIMAL(10,2) NOT NULL CHECK (ordered_quantity > 0),
    delivery_value   DECIMAL(12,2) NOT NULL CHECK (delivery_value >= 0),
    slot_start       DATETIME      NOT NULL,
    slot_end         DATETIME      NOT NULL,
    status           VARCHAR(20)   NOT NULL DEFAULT 'SCHEDULED'
                     CHECK (status IN ('SCHEDULED', 'PREPARING', 'IN_TRANSIT', 'DELIVERED',
'INSPECTED', 'COMPLETED')),
    FOREIGN KEY (project_id)  REFERENCES project (project_id),
    FOREIGN KEY (org_id)      REFERENCES organization (org_id),
    FOREIGN KEY (material_id) REFERENCES material (material_id),
    FOREIGN KEY (created_by)  REFERENCES user_account (user_id)
);
```

Result:

```
Query OK, 0 rows affected
```

1.11 Quality documents (PDF/JPG) uploaded to a delivery.

```
CREATE TABLE delivery_document (
    document_id INT AUTO_INCREMENT PRIMARY KEY,
    delivery_id INT          NOT NULL,
    uploaded_by INT          NOT NULL,
    doc_type    VARCHAR(50)  NOT NULL,
    file_name   VARCHAR(200) NOT NULL,
    FOREIGN KEY (delivery_id) REFERENCES delivery (delivery_id),
    FOREIGN KEY (uploaded_by) REFERENCES user_account (user_id)
);
```

Result:

```
Query OK, 0 rows affected
```

1.12 Inspection of a delivery (only one per delivery). The accepted quantity cannot be more than the received quantity.

```
CREATE TABLE inspection (
    inspection_id     INT AUTO_INCREMENT PRIMARY KEY,
    delivery_id       INT           NOT NULL UNIQUE,
    inspector_id      INT           NOT NULL,
    received_quantity DECIMAL(10,2) NOT NULL,
    accepted_quantity DECIMAL(10,2) NOT NULL,
    retention_rate    DECIMAL(4,3)  NOT NULL,
    CHECK (accepted_quantity >= 0 AND accepted_quantity <= received_quantity),
    FOREIGN KEY (delivery_id)  REFERENCES delivery (delivery_id),
    FOREIGN KEY (inspector_id) REFERENCES user_account (user_id)
);
```

Result:

```
Query OK, 0 rows affected
```

1.13 Mandatory 1-5 rating of the supplier when the delivery is completed (one per delivery).

```
CREATE TABLE supplier_rating (
    delivery_id INT     PRIMARY KEY,
    rated_by    INT     NOT NULL,
    score       TINYINT NOT NULL CHECK (score BETWEEN 1 AND 5),
    FOREIGN KEY (delivery_id) REFERENCES delivery (delivery_id),
    FOREIGN KEY (rated_by)    REFERENCES user_account (user_id)
);
```

Result:

```
Query OK, 0 rows affected
```

1.14 View that calculates the total price of each purchase: quantity x unit price x (1 - discount).

```
CREATE VIEW purchase_total AS
SELECT purchase_id,
       ROUND(quantity * unit_price * (100 - discount_rate) / 100, 2) AS total_price
FROM purchase;
```

Result:

```
Query OK, 0 rows affected
```

1.15 Check: list the tables created.

```
SHOW TABLES;
```

Result:

```
+---------------------+
| Tables_in_siteproof |
+---------------------+
| delivery            |
| delivery_document   |
| inspection          |
| material            |
| material_category   |
| organization        |
| project             |
| purchase            |
| purchase_total      |
| supplier_rating     |
| user_account        |
+---------------------+
11 rows in set
```

Script 2: Extract: load the files without changes

2.1 Raw table for companies.tsv (only the columns we need are used later).

```
CREATE TABLE raw_companies (
    company_id  VARCHAR(20),
    name        VARCHAR(200),
    description VARCHAR(200),
    address     VARCHAR(200),
    phone       VARCHAR(50),
    email       VARCHAR(100),
    comments    VARCHAR(200)
);
```

Result:

```
Query OK, 0 rows affected
```

2.2 Raw table for materials.tsv.

```
CREATE TABLE raw_materials (
    material_id VARCHAR(20),
    item        VARCHAR(300),
    unit        VARCHAR(20),
    price_avg   VARCHAR(20),
    price_stdev VARCHAR(20),
    no_samples  VARCHAR(20)
);
```

Result:

```
Query OK, 0 rows affected
```

2.3 Raw table for transactions.tsv.

```
CREATE TABLE raw_transactions (
    transaction_id   VARCHAR(20),
    company_id       VARCHAR(20),
    material_id      VARCHAR(20),
    quantity         VARCHAR(20),
    price_per_unit   VARCHAR(20),
    discount_rate    VARCHAR(20),
    total_price      VARCHAR(20),
    transaction_date VARCHAR(20),
    notes            VARCHAR(200)
);
```

Result:

```
Query OK, 0 rows affected
```

2.4 Load companies.tsv (columns separated by tabs, first line is the header).

```
LOAD DATA INFILE '/var/lib/mysql-files/companies.tsv'
INTO TABLE raw_companies
CHARACTER SET utf8mb4
FIELDS TERMINATED BY '\t'
LINES TERMINATED BY '\r\n'
IGNORE 1 LINES;
```

Result:

```
Query OK, 500 rows affected
Records: 500  Deleted: 0  Skipped: 0  Warnings: 0
```

2.5 Load materials.tsv.

```
LOAD DATA INFILE '/var/lib/mysql-files/materials.tsv'
INTO TABLE raw_materials
CHARACTER SET utf8mb4
FIELDS TERMINATED BY '\t'
LINES TERMINATED BY '\r\n'
IGNORE 1 LINES;
```

Result:

```
Query OK, 458 rows affected
Records: 458  Deleted: 0  Skipped: 0  Warnings: 0
```

2.6 Load transactions.tsv.

```
LOAD DATA INFILE '/var/lib/mysql-files/transactions.tsv'
INTO TABLE raw_transactions
CHARACTER SET utf8mb4
FIELDS TERMINATED BY '\t'
LINES TERMINATED BY '\r\n'
IGNORE 1 LINES;
```

Result:

```
Query OK, 51947 rows affected
Records: 51947  Deleted: 0  Skipped: 0  Warnings: 0
```

2.7 Check that all the rows were loaded (500 companies, 458 materials, 51,947 transactions).

```
SELECT (SELECT COUNT(*) FROM raw_companies)    AS companies,
       (SELECT COUNT(*) FROM raw_materials)    AS materials,
       (SELECT COUNT(*) FROM raw_transactions) AS transactions;
```

Result:

```
+-----------+-----------+--------------+
| companies | materials | transactions |
+-----------+-----------+--------------+
|       500 |       458 |        51947 |
+-----------+-----------+--------------+
1 row in set
```

Script 3: Find the errors

3.1 Error 1: transactions with a company_id that does not exist in the companies file (LEFT JOIN + IS NULL).

```
SELECT COUNT(*) AS unknown_company
FROM raw_transactions t
LEFT JOIN raw_companies c ON t.company_id = c.company_id
WHERE c.company_id IS NULL;
```

Result:

```
+-----------------+
| unknown_company |
+-----------------+
|            2615 |
+-----------------+
1 row in set
```

3.2 Error 2: transactions with a material_id that does not exist in the materials file.

```
SELECT COUNT(*) AS unknown_material
FROM raw_transactions t
LEFT JOIN raw_materials m ON t.material_id = m.material_id
WHERE m.material_id IS NULL;
```

Result:

```
+------------------+
| unknown_material |
+------------------+
|             2643 |
+------------------+
1 row in set
```

3.3 Error 3: transactions per decade. Almost all are from 2010-2025; the dates from 1950-1980 are not possible.

```
SELECT CONCAT(LEFT(transaction_date, 3), '0s') AS decade, COUNT(*) AS transactions
FROM raw_transactions
GROUP BY decade
ORDER BY decade;
```

Result:

```
+--------+--------------+
| decade | transactions |
+--------+--------------+
| 1950s  |         1670 |
| 1960s  |         1678 |
| 1970s  |         1674 |
| 1980s  |          176 |
| 2010s  |        12236 |
| 2020s  |        34513 |
+--------+--------------+
6 rows in set
```

3.4 Error 4: total_price different from quantity x price x (1 - discount). A difference up to 5 is allowed because the file rounds totals to tens.

```
SELECT COUNT(*) AS wrong_total
FROM raw_transactions
WHERE ABS(total_price - quantity * price_per_unit * (100 - discount_rate) / 100) > 5;
```

Result:

```
+-------------+
| wrong_total |
+-------------+
|        2535 |
+-------------+
1 row in set
```

3.5 Error 4 in detail: in all those rows the total is just a copy of the unit price.

```
SELECT transaction_id, quantity, price_per_unit, discount_rate, total_price
FROM raw_transactions
WHERE ABS(total_price - quantity * price_per_unit * (100 - discount_rate) / 100) > 5
LIMIT 5;
```

Result:

```
+----------------+----------+----------------+---------------+-------------+
| transaction_id | quantity | price_per_unit | discount_rate | total_price |
+----------------+----------+----------------+---------------+-------------+
| 66227135       | 149      | 25000          | 0             | 25000       |
| 93888842       | 21       | 100            | 0             | 100         |
| 22089478       | 418      | 100            | 16            | 100         |
| 81613956       | 265      | 5600           | 5             | 5600        |
| 66106919       | 367      | 900            | 5             | 900         |
+----------------+----------+----------------+---------------+-------------+
5 rows in set
```

3.6 Error 5: the same material twice (same name and unit) with two different ids.

```
SELECT item, unit, COUNT(*) AS times, GROUP_CONCAT(material_id) AS ids
FROM raw_materials
GROUP BY item, unit
HAVING COUNT(*) > 1;
```

Result:

```
+-----------------------------------------------------+-------+-------+-------------------+
| item                                                | unit  | times | ids               |
+-----------------------------------------------------+-------+-------+-------------------+
| Products. Graded aggregate base. Graded crushed stone | L.m³  |     2 | 51019678,65579548 |
+-----------------------------------------------------+-------+-------+-------------------+
1 row in set
```

3.7 Error 6: units written in two ways ("piece" and "Piece", "one test" and "One test"). COLLATE utf8mb4_bin shows upper and lower case as different.

```
SELECT unit COLLATE utf8mb4_bin AS unit, COUNT(*) AS materials
FROM raw_materials
WHERE unit IN ('piece', 'one test')
GROUP BY unit COLLATE utf8mb4_bin;
```

Result:

```
+----------+-----------+
| unit     | materials |
+----------+-----------+
| Piece    |        89 |
| piece    |         8 |
| One test |        13 |
| one test |         4 |
+----------+-----------+
4 rows in set
```

Script 4: Transform and load

4.1 Fix error 1: delete the transactions whose company does not exist.

```
DELETE FROM raw_transactions
WHERE company_id NOT IN (SELECT company_id FROM raw_companies);
```

Result:

```
Query OK, 2615 rows affected
```

4.2 Fix error 2: delete the transactions whose material does not exist.

```
DELETE FROM raw_transactions
WHERE material_id NOT IN (SELECT material_id FROM raw_materials);
```

Result:

```
Query OK, 2497 rows affected
```

4.3 Fix error 3: delete the transactions with an impossible date (before 2000).

```
DELETE FROM raw_transactions
WHERE transaction_date < '2000-01-01';
```

Result:

```
Query OK, 4705 rows affected
```

4.4 Fix error 5 (part 1): move the transactions of the duplicated material 65579548 to the material we keep, 51019678 (it has more price samples).

```
UPDATE raw_transactions
SET material_id = '51019678'
WHERE material_id = '65579548';
```

Result:

```
Query OK, 83 rows affected
Rows matched: 83  Changed: 83  Warnings: 0
```

4.5 Fix error 5 (part 2): delete the duplicated material.

```
DELETE FROM raw_materials
WHERE material_id = '65579548';
```

Result:

```
Query OK, 1 row affected
```

4.6 Fix error 6: write the units always the same way. The comparison ignores upper/lower case, so both spellings get the same value.

```
UPDATE raw_materials SET unit = 'Piece'    WHERE unit = 'piece';
```

Result:

```
Query OK, 8 rows affected
Rows matched: 97  Changed: 8  Warnings: 0
```

```
UPDATE raw_materials SET unit = 'One test' WHERE unit = 'one test';
```

Result:

```
Query OK, 4 rows affected
Rows matched: 17  Changed: 4  Warnings: 0
```

4.7 Create the categories.

```
INSERT INTO material_category (category_name)
VALUES ('Labor'), ('Equipment'), ('Material'), ('Other');
```

Result:

```
Query OK, 4 rows affected
Records: 4  Duplicates: 0  Warnings: 0
```

4.8 Load the companies. All of them are suppliers. Only the columns we need are used.

```
INSERT INTO organization (org_id, org_name, org_type, address, phone, email)
SELECT company_id, name, 'SUPPLIER', address, phone, email
FROM raw_companies;
```

Result:

```
Query OK, 500 rows affected
Records: 500  Duplicates: 0  Warnings: 0
```

4.9 Load the materials and give each one a category: "Product..." items are materials; items paid by hour/day/trip are equipment if they mention a machine, otherwise labor; the rest is other.

```
INSERT INTO material (material_id, material_name, unit, category_id)
SELECT material_id,
       item,
       unit,
       CASE
         WHEN item LIKE 'Product%' THEN 3
         WHEN unit IN ('Hour', 'Day', 'Trip')
              AND (item LIKE '%machine%' OR item LIKE '%equipment%' OR item LIKE '%crawler%'
                   OR item LIKE '%wheeled%' OR item LIKE '%trailer%' OR item LIKE '%crusher%') THEN 2
         WHEN unit IN ('Hour', 'Day') THEN 1
         ELSE 4
       END
FROM raw_materials;
```

Result:

```
Query OK, 457 rows affected
Records: 457  Duplicates: 0  Warnings: 0
```

4.10 Load the clean transactions as purchases. Fix error 4: total_price is not copied; the view purchase_total calculates it again.

```
INSERT INTO purchase (purchase_id, org_id, material_id, quantity, unit_price, discount_rate,
purchase_date)
SELECT transaction_id, company_id, material_id, quantity, price_per_unit, discount_rate,
transaction_date
FROM raw_transactions;
```

Result:

```
Query OK, 42130 rows affected
Records: 42130  Duplicates: 0  Warnings: 0
```

Script 5: Check the result

5.1 Rows in the final tables (51,947 transactions - 9,817 deleted = 42,130 purchases; 458 - 1 duplicated = 457 materials).

```
SELECT (SELECT COUNT(*) FROM organization) AS companies,
       (SELECT COUNT(*) FROM material)     AS materials,
       (SELECT COUNT(*) FROM purchase)     AS purchases;
```

Result:

```
+-----------+-----------+-----------+
| companies | materials | purchases |
+-----------+-----------+-----------+
|       500 |       457 |     42130 |
+-----------+-----------+-----------+
1 row in set
```

5.2 No purchase points to a company or material that does not exist (both must be 0).

```
SELECT (SELECT COUNT(*) FROM purchase p LEFT JOIN organization o ON p.org_id = o.org_id WHERE o.org_id
IS NULL)          AS purchases_without_company,
       (SELECT COUNT(*) FROM purchase p LEFT JOIN material m ON p.material_id = m.material_id WHERE
m.material_id IS NULL) AS purchases_without_material;
```

Result:

```
+--------------------------+--------------------------+
| purchases_without_company | purchases_without_material |
+--------------------------+--------------------------+
|                         0 |                        0 |
+--------------------------+--------------------------+
1 row in set
```

5.3 Dates: the oldest and newest purchase.

```
SELECT MIN(purchase_date) AS first_date, MAX(purchase_date) AS last_date
FROM purchase;
```

Result:

```
+------------+------------+
| first_date | last_date  |
+------------+------------+
| 2018-01-01 | 2025-07-31 |
+------------+------------+
1 row in set
```

5.4 Error 4 fixed: rows where the original total was wrong now have the correct total from the view.

```
SELECT p.purchase_id, r.total_price AS wrong_total_in_file, v.total_price AS correct_total
FROM purchase p
JOIN purchase_total v   ON p.purchase_id = v.purchase_id
JOIN raw_transactions r ON p.purchase_id = r.transaction_id
WHERE ABS(r.total_price - v.total_price) > 5
LIMIT 5;
```

Result:

```
+-------------+--------------------+---------------+
| purchase_id | wrong_total_in_file | correct_total |
+-------------+--------------------+---------------+
| 66227135    | 25000              |    3725000.00 |
| 93888842    | 100                |       2100.00 |
| 22089478    | 100                |      35112.00 |
| 81613956    | 5600               |    1409800.00 |
| 66106919    | 900                |     313785.00 |
+-------------+--------------------+---------------+
5 rows in set
```

5.5 Units after cleaning (no more "piece" / "one test").

```
SELECT unit COLLATE utf8mb4_bin AS unit, COUNT(*) AS materials
FROM material
GROUP BY unit COLLATE utf8mb4_bin
ORDER BY materials DESC;
```

Result:

```
+------------+-----------+
| unit       | materials |
+------------+-----------+
| Piece      |        97 |
| m³         |        68 |
| m          |        57 |
| m²         |        56 |
| kg         |        46 |
| Day        |        37 |
| Hour       |        22 |
| One test   |        17 |
| Metric Ton |         9 |
| L.m³       |         9 |
| L          |         8 |
| root       |         6 |
| Pair       |         6 |
| Set        |         5 |
| Bag        |         4 |
| B.m³       |         3 |
| Trip       |         2 |
| 900cm²     |         2 |
| C.m³       |         2 |
| km         |         1 |
+------------+-----------+
20 rows in set
```

5.6 Materials per category.

```
SELECT c.category_name, COUNT(*) AS materials
FROM material m
JOIN material_category c ON m.category_id = c.category_id
GROUP BY c.category_name;
```

Result:

```
+---------------+-----------+
| category_name | materials |
+---------------+-----------+
| Labor         |        35 |
| Equipment     |        26 |
| Material      |       313 |
| Other         |        83 |
+---------------+-----------+
4 rows in set
```

5.7 The constraints protect the data: this insert has an impossible date, so it must FAIL (expected error).

```
INSERT INTO purchase (purchase_id, org_id, material_id, quantity, unit_price, discount_rate,
purchase_date)
VALUES ('TEST0001', '03028108', '51019678', 10, 100, 0, '1959-06-29');
```

Result:

```
ERROR 3819 (HY000) at line 48: Check constraint 'purchase_chk_4' is violated.
```

5.8 This insert uses a company that does not exist, so it must FAIL (expected error).

```
INSERT INTO purchase (purchase_id, org_id, material_id, quantity, unit_price, discount_rate,
purchase_date)
VALUES ('TEST0002', '99999999', '51019678', 10, 100, 0, '2024-01-15');
```

Result:

```
ERROR 1452 (23000) at line 53: Cannot add or update a child row: a foreign key constraint fails
(`siteproof`.`purchase`, CONSTRAINT `purchase_ibfk_1` FOREIGN KEY (`org_id`) REFERENCES `organization`
(`org_id`))
```

5.9 Example: the 5 suppliers we bought the most from.

```
SELECT o.org_name, COUNT(*) AS purchases, ROUND(SUM(v.total_price)) AS total_amount
FROM purchase p
JOIN purchase_total v ON p.purchase_id = v.purchase_id
JOIN organization o   ON p.org_id = o.org_id
GROUP BY o.org_id, o.org_name
ORDER BY total_amount DESC
LIMIT 5;
```

Result:

```
+------------------------------+-----------+--------------+
| org_name                     | purchases | total_amount |
+------------------------------+-----------+--------------+
| Riverbend Construction       |        99 |    146009809 |
| Riverbend Supply             |        98 |    113599203 |
| Millennium Timber Enterprises |        91 |    100733753 |
| Liberty Scaffolding          |        86 |     98039361 |
| Horizon Logistics Enterprises |        91 |     96572826 |
+------------------------------+-----------+--------------+
5 rows in set
```